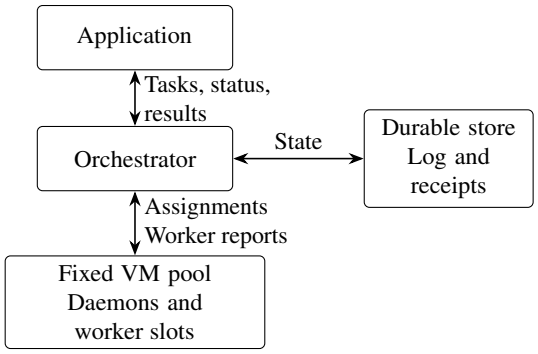


Figure 1: Durable Swarms separates disposable worker execution from durable coordination state. Worker updates and simulated effects pass through the orchestrator, which validates the current attempt before committing changes.

1 System Architecture

Durable Swarms coordinates independent, concurrent workflows over a fixed pool of virtual machines (VMs). Our objective is to preserve completed work across worker failures while separating application logic from placement, monitoring, and recovery. We instantiate the application layer with simulated workflows that model a leader and its subtasks. This choice allows us to examine execution and recovery mechanisms without depending on live language-model agents; it does not model semantic agent misalignment or establish the quality of generated code.

1.1 Execution Model and Components

A workload consists of one logical leader and a set of subtask workers. Each worker executes an ordered sequence of steps, and dependencies between subtasks form a directed acyclic graph. The current specification declares the subtasks, their dependencies, and expected results at submission time. Steps represent compute delays or simulated side effects. A seed determines any sampled delays; the normalized specification and fault schedule are persisted with the workload. These inputs are reproducible, although concurrent scheduling and elapsed execution time remain timing-dependent.

Figure 1 shows the three runtime components: a single orchestrator, a durable store, and VM daemons that host disposable worker processes. The application submits a specification and retrieves status and results through an HTTP interface. It does not assign workers to machines or reconstruct them after a failure. The prototype exposes a declarative workflow interface; a general interface for programmer-supplied execution functions remains future work.

Each VM has a configured number of worker slots. Its daemon registers a session, polls for assignments, launches worker subprocesses, and reports their liveness. The orchestrator owns placement and infrastructure failure detection. A leader creates its declared children through the orchestrator, collects their results, and authorizes retries for failed children. Thus, infrastructure detection is centralized while child-recovery decisions remain associated with the parent workload. Recovery of the leader itself is initiated by the orchestrator.

1.2 Placement and Coordination

The scheduler assigns workers whose dependencies have completed to available slots on live VMs. Within a scheduling cycle, it considers at most one ready worker per workload per pass and repeats while capacity remains. Placement prefers a VM other than the worker's previous failed placement, then a VM with lower slot occupancy relative to its capacity. Migration is therefore preferred when another suitable VM is available, rather than required for every recovery. Slot accounting bounds assigned concurrency; it does not measure or enforce per-worker CPU or memory consumption.

A leader that has completed its own steps releases its slot while awaiting child results. Child failure or completion of all children makes the leader runnable again. This prevents a waiting leader from occupying the only slot needed to execute its subtasks. Completion requires every declared worker to finish and the leader to aggregate their results.

1.3 Durable State

The durable store uses SQLite with write-ahead logging and full synchronization. An event records its sequence number, workload identifier, timestamp, state version, event type, and payload. Worker-specific events additionally identify the logical worker, parent, and execution attempt. The history includes assignments, progress, subtask creation, effect intents and receipts, completion, and recovery transitions. State-changing workload transactions also append a snapshot containing worker state and each worker's next incomplete step.

The workload table is a materialized projection of these snapshots. On orchestrator restart, the projection is reconstructed from the checkpoint history. Changes to a workload, its events, and its snapshot commit in one transaction before acknowledgment. Worker VMs therefore need no durable local execution state. Recovery skips completed steps, but an interrupted compute delay may execute again because its completion has not yet been checkpointed.

1.4 Failure Detection and Recovery

We distinguish VM liveness from worker progress. A VM heartbeat renews the VM lease and only those unexpired worker attempts that its daemon reports as alive. A separate progress timeout detects workers that remain alive but stop advancing. Repeated reports of the same progress value do not reset this timer. Timeouts can also suspect delayed or

disconnected workers, so recovery must remain safe even when an old process continues running.

Each assignment receives a monotonically increasing attempt identifier. Worker requests carry the workload and worker identifiers, attempt identifier, VM identifier, and VM session. Before accepting a mutation, the orchestrator checks this identity against the current assignment and verifies that both leases remain valid. Superseded attempts cannot update progress, commit checkpoints, or apply simulated effects through the service.

When a failure is detected, the orchestrator invalidates the current attempt and retains its durable step cursor. A failed leader is queued for replacement; a failed child awaits retry authorization from its leader. The scheduler then assigns a replacement attempt using the retained checkpoint. Invalid final results fail the workload, while exhaustion of the configured failure budget escalates it. These outcomes make unsuccessful recovery explicit rather than reporting an incomplete workload as successful.

1.5 Replay and Side-Effect Safety

Checkpointing alone cannot prevent duplicate effects: a worker may apply an operation and fail before recording step completion. We therefore give each simulated effect a stable identifier, $o = H(w,a,s)$, derived from workload identifier w, logical worker identifier a, and step index s. The identifier excludes the execution attempt and VM, so a replacement worker refers to the same logical operation.

Execution proceeds in three stages. First, the worker records a durable intent for the effect. Second, the service applies the simulated mutation and stores its receipt atomically; a repeated operation identifier returns the original receipt. Third, the worker advances its step checkpoint after the receipt is durable. A failure before mutation leaves the operation unapplied. A failure after mutation but before checkpointing causes the replacement to retrieve the existing receipt without repeating the mutation. Once the checkpoint commits, the completed step is skipped during recovery.

Simulated effects increment counters scoped to each workload. The counter mutation, receipt, and completion event share a SQLite transaction, which closes the ambiguity window for this simulated destination. This provides at-least-once step execution with idempotent effect application, not a general exactly-once guarantee for arbitrary external tools. An external destination would require its own idempotency or reconciliation mechanism. Inconsistent effect records detected by the service cause escalation rather than automatic reapplication.

1.6 Scope and Current Tradeoffs

The design assumes a trusted private network, one active orchestrator, and durable storage that survives process restarts. Worker crashes, VM-daemon loss, stalls, retry loops, invalid results, and interruption after an effect are represented by the simulator. Loss of the orchestrator's underlying storage and Byzantine workers are outside the recovery model. Resetting a VM registration fences its assignments and permits a new daemon session; provisioning or booting VMs remains the responsibility of the existing VM manager.

The implementation favors explicit recovery state over optimized metadata handling. Progress and lease updates can write complete workload snapshots, and scheduling and monitoring scan workload records. Scheduling passes also restart in insertion order, so the current policy does not establish fairness across successive cycles. Reducing snapshot frequency, indexing active work, and retaining a rotating scheduling cursor are targeted refinements for the resource-efficiency goal. We do not claim large-cluster scalability or measured efficiency from the present architecture alone.